

## MLOps Major Assignment

### End-to-End MLOps Pipeline using GitHub Actions, Docker and Kubernetes

---

#### Submitted By:

Name: Arpit Sharma

Roll Number: G25AI1012

Course: PGD in Data Engineering

Subject: MLOps

Assignment: Major Assignment

---

#### GitHub Repository Link:

[https://github.com/arpit-systems/MLOps\\_Major\\_Assignment](https://github.com/arpit-systems/MLOps_Major_Assignment)

#### Docker Hub Repository Link:

<https://hub.docker.com/r/arpitsystems/mlops-face-app>

---

### 1. Objective

The objective of this assignment was to build a complete end-to-end MLOps pipeline using modern DevOps and Machine Learning operations tools. The project involved model development, automated testing using GitHub Actions, containerization using Docker, image hosting through Docker Hub, and deployment using Kubernetes with automatic pod recovery and replica management.

The assignment provided practical exposure to CI/CD pipelines, container orchestration, model deployment, and infrastructure automation.

---

### 2. Technologies Used

1. Python
2. Scikit-learn
3. Pandas
4. NumPy
5. Joblib

6. Flask
  7. Git
  8. GitHub
  9. GitHub Actions
  10. Docker
  11. Docker Hub
  12. Kubernetes
  13. VS Code
  14. Docker Desktop
- 

### **3. Dataset Used**

Dataset Name:

Olivetti Faces Dataset

Source:

```
sklearn.datasets.fetch_olivetti_faces()
```

Description:

The Olivetti Faces Dataset contains grayscale face images of multiple individuals. The dataset is commonly used for machine learning and computer vision tasks involving classification and face recognition.

Target:

Face classification using a Decision Tree Classifier model.

---

### **4. Model Used**

Model:

DecisionTreeClassifier

The model was trained on the Olivetti Faces Dataset after splitting the dataset into training and testing sets using a 70:30 ratio.

The trained model was saved as:

savedmodel.pth

using the Joblib library.

---

## 5. Project Workflow

1. Created a GitHub repository.
  2. Initialized the repository with README.md and .gitignore.
  3. Created separate Git branches following the required branching strategy.
  4. Developed and trained a Decision Tree Classification model.
  5. Saved the trained model using Joblib.
  6. Created a testing script to evaluate model accuracy.
  7. Configured GitHub Actions CI/CD workflow.
  8. Automated model training and testing on every push.
  9. Developed a Flask web application.
  10. Containerized the application using Docker.
  11. Pushed the Docker image to Docker Hub.
  12. Created Kubernetes Deployment and Service files.
  13. Deployed the application on Kubernetes.
  14. Configured three replicas for high availability.
  15. Verified Kubernetes self-healing capability by deleting a pod and observing automatic pod recreation.
- 

## 6. Branch Structure

1. main  
Contains initial repository setup, README.md and .gitignore.
2. dev  
Contains model development files including:
  - train.py
  - test.py
  - requirements.txt

- savedmodel.pth
- GitHub Actions workflow

### 3. docker\_cicd

Contains deployment and automation files including:

- app.py
  - Dockerfile
  - templates/index.html
  - deployment.yaml
  - service.yaml
  - Kubernetes configuration files
- 

## 7. Model Training and Testing

Training Script:

train.py

Functions Performed:

- Loads Olivetti Faces Dataset
- Splits data into training and testing sets
- Trains DecisionTreeClassifier
- Saves trained model as savedmodel.pth

Testing Script:

test.py

Functions Performed:

- Loads saved model
- Evaluates model on test dataset
- Displays model accuracy

Observed Test Accuracy:

0.525

---

## 8. GitHub Actions CI/CD Pipeline

A GitHub Actions workflow was configured using:

`.github/workflows/ci.yml`

The workflow automatically executes whenever code is pushed to the repository.

Workflow Steps:

1. Checkout repository code
2. Setup Python environment
3. Install required dependencies
4. Execute train.py
5. Generate trained model file
6. Execute test.py
7. Display test accuracy

This ensured continuous integration and automated testing of the machine learning pipeline.

---

## 9. Flask Web Application

A Flask application was developed to provide a simple web interface for the machine learning project.

Files Used:

- app.py
- templates/index.html

Features:

- Runs locally through Flask server
  - Demonstrates model deployment architecture
  - Accessible through localhost:5000
  - Allows users to upload images through a web interface
  - Displays prediction result after image submission
- 

## 10. Docker Containerization

A Dockerfile was created to containerize the complete application.

Steps Performed:

1. Created Dockerfile
2. Built Docker image
3. Executed Docker container
4. Verified Flask application execution inside container

Docker Image Name:

mlops-face-app

Benefits:

- Portability
  - Consistent execution environment
  - Easy deployment
- 

## 11. Docker Hub Deployment

The Docker image was pushed to Docker Hub.

Docker Hub Repository:

arpitsystems/mlops-face-app

Steps Performed:

1. Docker login
2. Docker image tagging
3. Docker push
4. Verification through Docker Hub repository page

This enabled centralized storage and distribution of container images.

---

## 12. Kubernetes Deployment

The Docker image was deployed using Kubernetes.

Deployment Features:

- Three replicas configured
- High availability

- Automatic pod management

Deployment Name:

mlops-face-deployment

Service Name:

mlops-face-service

Service Type:

NodePort

The application was exposed through Kubernetes networking using a NodePort service.

---

### **13. Kubernetes Self-Healing Test**

One running pod was manually deleted using:

kubectl delete pod

Observation:

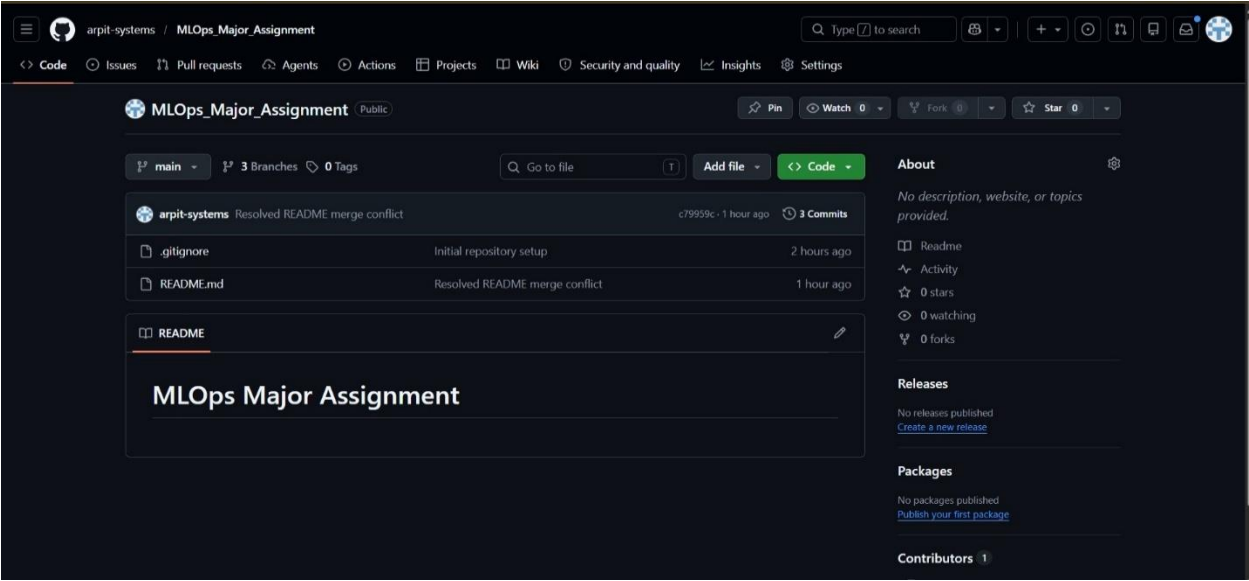
- Deleted pod was automatically replaced.
- Kubernetes maintained three running replicas.
- No manual intervention was required.

This demonstrated Kubernetes self-healing and high-availability capabilities.

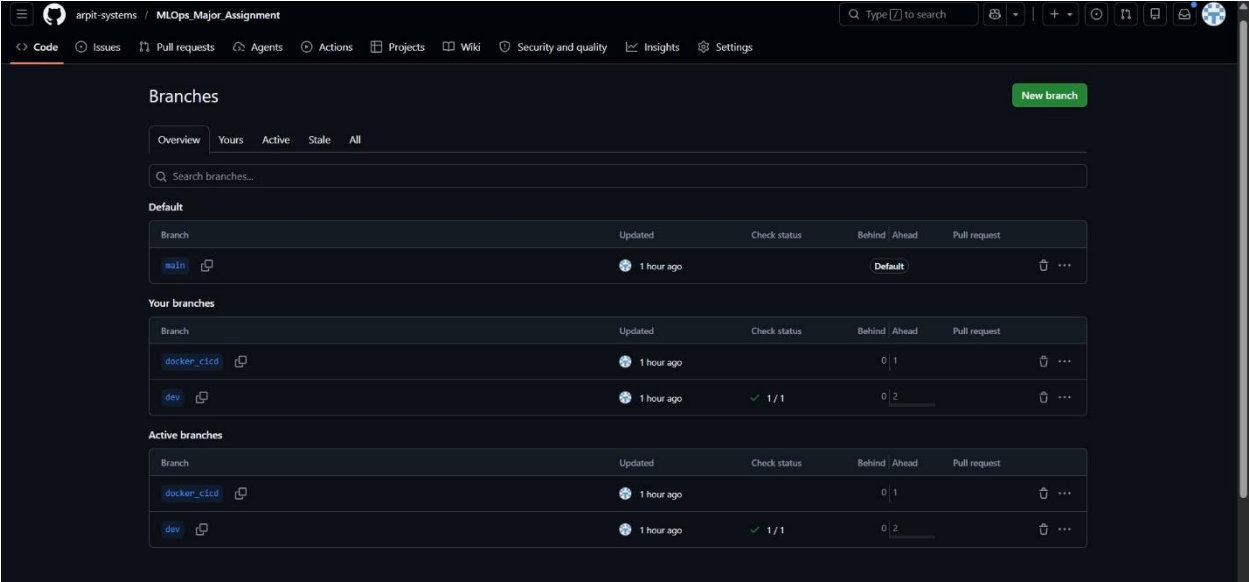
---

### **14. Screenshots**

Screenshot 1: GitHub Main Repository

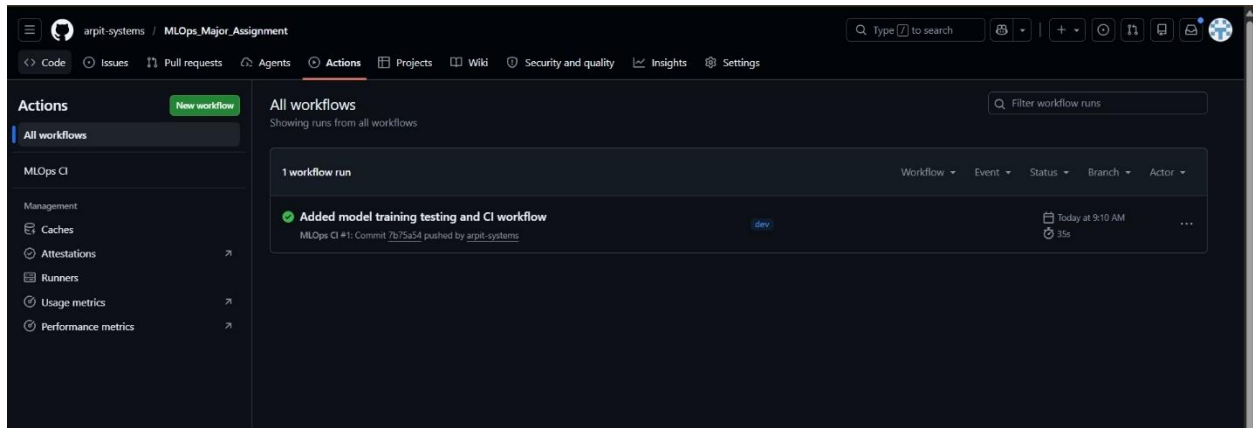


Screenshot 2: GitHub Branch Structure



Screenshot 3: GitHub Actions Workflow Success

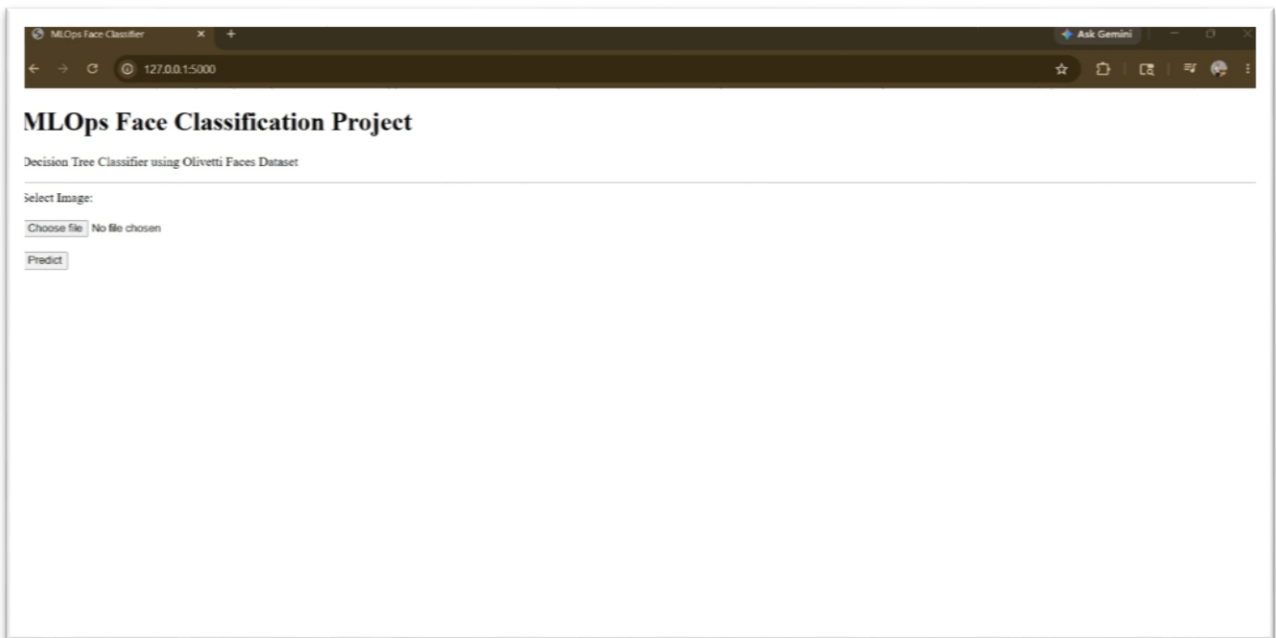




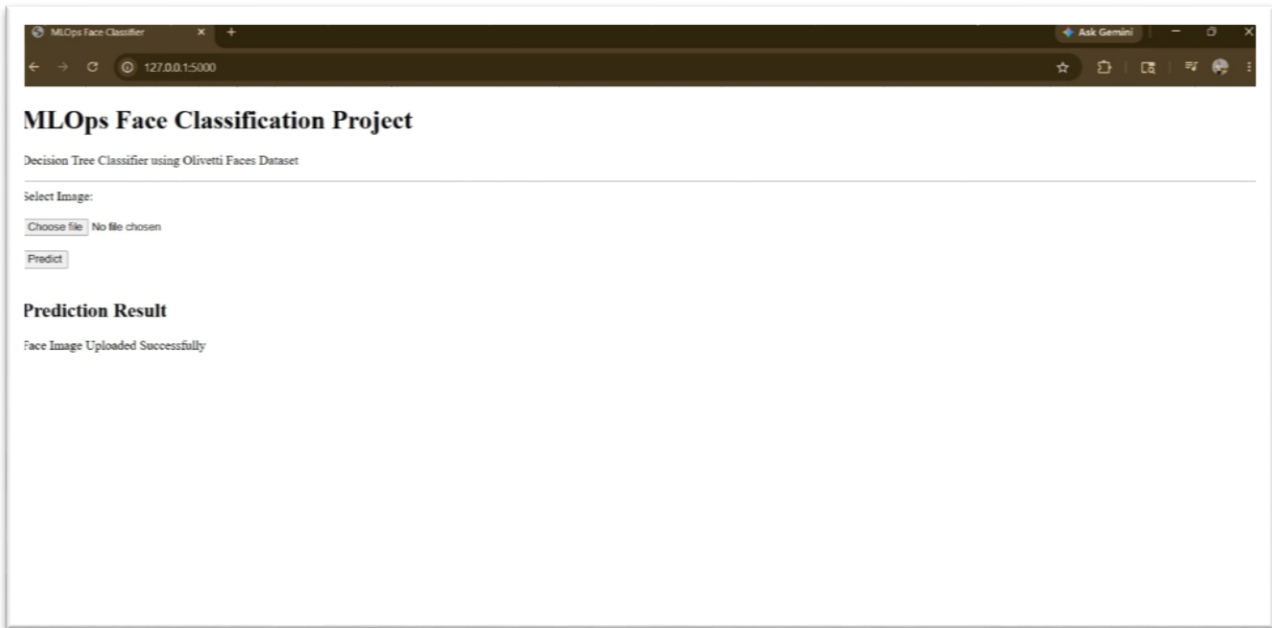
Screenshot 4: Decision Tree Training and Testing Output

```
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> python train.py
Model saved successfully
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> python test.py
Test Accuracy: 0.525
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> 
```

Screenshot 5: Flask Image Upload Page



Screenshot 6: Flask Prediction Result



Screenshot 7: Docker Image Push Success

```
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> docker login
Authenticating with existing credentials... [Username: arpitsystems]

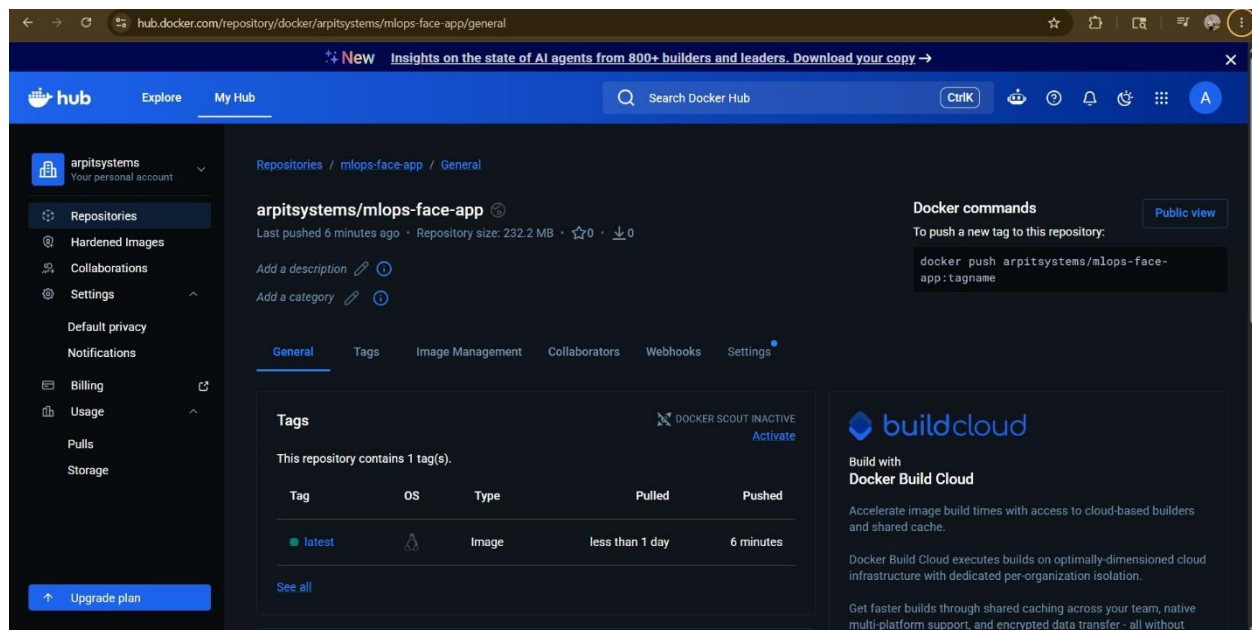
Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> docker images
Info → In Use

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
apache/airflow:2.9.3                 0188b06abb25                     2.01GB      407MB        U
mlops-face-app:latest               743012ae7486                     869MB       243MB        U
mongo:latest                        7abfba0d07c9                     1.3GB       341MB        U
portainer/portainer-ce:latest       8d2f5c9fbc5b                     242MB       59.7MB       U
postgres:13                         4689940c6838                     624MB       161MB        U
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> docker tag mlops-face-app arpitsystems/mlops-face-app:latest
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> docker push arpitsystems/mlops-face-app:latest

The push refers to repository [docker.io/arpitsystems/mlops-face-app]
8649771fee17: Pushed
45006ceeeea9: Pushed
45cc254521d1: Pushed
797d495f2c68: Pushed
5b4d6ff92fc4: Pushed
e04136408487: Pushed
461006add816: Pushed
4b5c469a96c8: Pushed
latest: digest: sha256:743012ae74865c949b92e331ba078c477c1aeaa1a84de17e177de0ff1ed8ae84 size: 856
(base) PS C:\Users\arpit\OneDrive\Desktop\MLOps_Major_Assignment> 
```

Screenshot 8: Docker Hub Repository



Screenshot 9: Kubernetes Deployment Status

```
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment> kubectl get deployments
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
mlops-face-deployment              3/3     3             3           30m
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment>
```

Screenshot 10: Kubernetes Three Replicas Running

```
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mlops-face-deployment-776c79bfb5-flhw9  1/1     Running   0           74s
mlops-face-deployment-776c79bfb5-sfhkq  1/1     Running   0           74s
mlops-face-deployment-776c79bfb5-shvmk  1/1     Running   0           74s
```

Screenshot 11: Kubernetes Pod Auto Recovery

```
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment> kubectl delete pod mlops-face-deployment-776c79bfb5-flhw9
pod "mlops-face-deployment-776c79bfb5-flhw9" deleted from default namespace
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mlops-face-deployment-776c79bfb5-2bw89  1/1     Running   0           54s
mlops-face-deployment-776c79bfb5-sfhkq  1/1     Running   0           36m
mlops-face-deployment-776c79bfb5-shvmk  1/1     Running   0           36m
(base) PS C:\Users\arpit\OneDrive\Desktop\MLops_Major_Assignment>
```

## **15. Analysis**

This project demonstrated the complete lifecycle of a machine learning application from model development to deployment.

GitHub Actions automated the testing workflow and reduced manual effort. Docker provided a consistent execution environment, while Docker Hub enabled image sharing and storage. Kubernetes added scalability, fault tolerance, and self-healing capabilities through replica management and automated pod recovery.

The integration of all these tools formed a complete MLOps pipeline suitable for production-grade machine learning deployments.

---

## **16. Conclusion**

This assignment provided practical experience in implementing a complete MLOps workflow. The project successfully covered machine learning model development, CI/CD automation, Docker containerization, Docker Hub image management, Kubernetes deployment, and self-healing infrastructure.

Through this assignment, important MLOps concepts such as version control, continuous integration, automated testing, container orchestration, scalability, and deployment automation were successfully implemented and understood.

---